Formal Logic (SM234614)

First Meeting: Boolean Logic

Muhammad Syifa'ul Mufid

Department of Mathematics
Institut Teknologi Sepuluh Nopember Surabaya

Boolean logic is an algebraic structure defined over a set

$$\mathbb{B} := \{0, 1\} \text{ or } \mathbb{B} := \{\texttt{false}, \texttt{true}\}.$$

Boolean logic is an algebraic structure defined over a set

$$\mathbb{B} := \{0, 1\} \text{ or } \mathbb{B} := \{\texttt{false}, \texttt{true}\}.$$

There are two operations in Boolean logic:

Boolean logic is an algebraic structure defined over a set

$$\mathbb{B} := \{0, 1\} \text{ or } \mathbb{B} := \{\texttt{false}, \texttt{true}\}.$$

There are two operations in Boolean logic:

- Conjunction (AND, $\wedge$)

$$\wedge : (\mathbb{B}, \mathbb{B}) \to \mathbb{B}$$

$a \wedge b$ yields 1 if both a and b are 1; 0 otherwise

Boolean logic is an algebraic structure defined over a set

$$\mathbb{B} := \{0, 1\} \text{ or } \mathbb{B} := \{\texttt{false}, \texttt{true}\}.$$

There are two operations in Boolean logic:

- Conjunction (AND, $\wedge$)

$$\wedge : (\mathbb{B}, \mathbb{B}) \to \mathbb{B}$$

$a \wedge b$ yields 1 if both a and b are 1; 0 otherwise

- Negations (NOT, $\neg$)

$$\neg : \mathbb{B} \to \mathbb{B}$$

$\neg 1 = 0$ and $\neg 0 = 1$

Other operations can be constructed using $\wedge$ dan $\neg$

Other operations can be constructed using $\wedge$ dan $\neg$

- Disjunction (OR, $\vee$)

$$a \vee b := \neg(\neg a \wedge \neg b)$$

$a \wedge b$ yields 0 if both a and b are 0; 1 otherwise

Other operations can be constructed using $\wedge$ dan $\neg$

- Disjunction (OR, $\vee$)

$$a \vee b := \neg(\neg a \wedge \neg b)$$

$a \wedge b$ yields 0 if both a and b are 0; 1 otherwise

- Implication (IF, $\rightarrow$)

$$a \rightarrow b := \neg(a \wedge \neg b)$$

$a \rightarrow b$ yields 0 if $a = 1$ and $b = 0$; 1 otherwise

Other operations can be constructed using $\wedge$ dan $\neg$

- Disjunction (OR, $\vee$)

$$a \vee b := \neg(\neg a \wedge \neg b)$$

$a \wedge b$ yields 0 if both a and b are 0; 1 otherwise

- Implication (IF, $\rightarrow$)

$$a \rightarrow b := \neg(a \wedge \neg b)$$

$a \rightarrow b$ yields 0 if $a = 1$ and $b = 0$; 1 otherwise

- Biimplication (IFF, $\leftrightarrow$)

$$a \leftrightarrow b := \neg(\neg(a \wedge b) \wedge \neg(\neg(a) \wedge \neg(b)))$$

$a \leftrightarrow b$ yields 1 if a and b have the same value; 0 otherwise

Other operations can be constructed using $\wedge$ dan $\neg$

- Disjunction (OR, $\vee$)

$$a \vee b := \neg(\neg a \wedge \neg b)$$

  $a \wedge b$ yields 0 if both a and b are 0; 1 otherwise

- Implication (IF, $\rightarrow$)

$$a \rightarrow b := \neg(a \wedge \neg b)$$

  $a \rightarrow b$ yields 0 if $a = 1$ and $b = 0$; 1 otherwise

- Biimplication (IFF, $\leftrightarrow$)

$$a \leftrightarrow b := \neg(\neg(a \wedge b) \wedge \neg(\neg(a) \wedge \neg(b)))$$

  $a \leftrightarrow b$ yields 1 if a and b have the same value; 0 otherwise

- Etc (NOR, XOR, ...)

For the sake of "simplicity", only three operations will be used: $\neg, \wedge$, dan $\vee$.

For the sake of "simplicity", only three operations will be used: $\neg, \wedge$, dan $\vee$. Additionally, we "define" $\wedge$ as multiplication and $\vee$ as addition

For the sake of "simplicity", only three operations will be used: $\neg, \wedge$, dan $\vee$. Additionally, we "define" $\wedge$ as multiplication and $\vee$ as addition

$$a \wedge b \equiv ab$$

For the sake of "simplicity", only three operations will be used: $\neg, \wedge$, dan $\vee$. Additionally, we "define" $\wedge$ as multiplication and $\vee$ as addition

$$a \wedge b \equiv ab$$
$$a \vee b \equiv a + b$$

For the sake of "simplicity", only three operations will be used: $\neg, \wedge$, dan $\vee$. Additionally, we "define" $\wedge$ as multiplication and $\vee$ as addition

$$a \wedge b \equiv ab$$
$$a \vee b \equiv a + b$$
$$a \to b \equiv \neg(a \wedge \neg b) \equiv \neg a \vee b \equiv \neg a + b$$

For the sake of "simplicity", only three operations will be used: $\neg, \wedge$, dan $\vee$. Additionally, we "define" $\wedge$ as multiplication and $\vee$ as addition

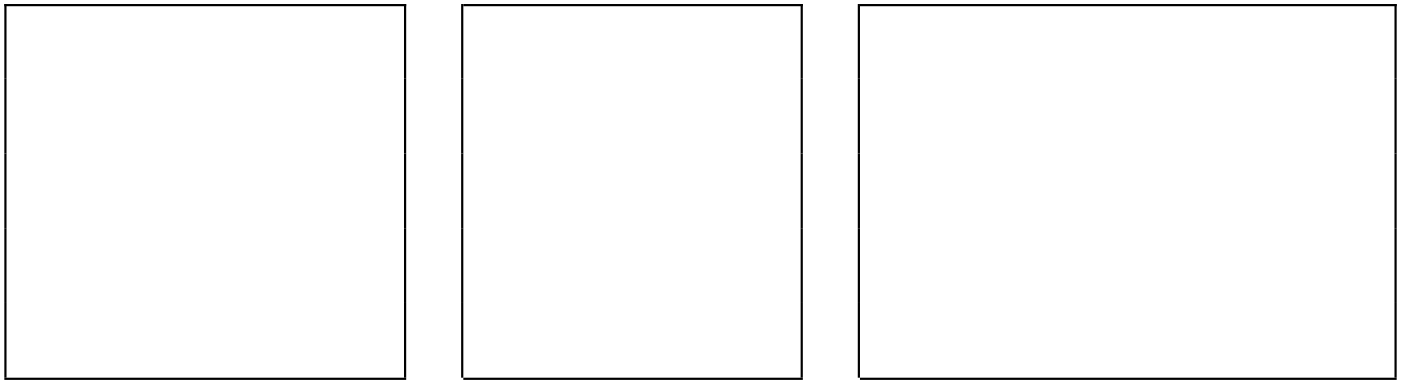
$$a \wedge b \equiv ab$$

$$a \vee b \equiv a + b$$

$$a \to b \equiv \neg(a \wedge \neg b) \equiv \neg a \vee b \equiv \neg a + b$$

$$a \leftrightarrow b \equiv \neg(\neg(a \wedge b) \wedge \neg(\neg(a) \wedge \neg(b))) \equiv ab + \neg a \neg b$$

Several properties



Several properties

$$a + 1 = 1$$
$$a + 0 = a$$
$$a + a = a$$
$$a + \neg a = 1$$
$$a + b = b + a$$

Several properties

$$a + 1 = 1$$
$$a + 0 = a$$
$$a + a = a$$
$$a + \neg a = 1$$
$$a + b = b + a$$

$$a \cdot 1 = a$$
$$a \cdot 0 = 0$$
$$a \cdot a = a$$
$$a \cdot \neg a = 0$$
$$a \cdot b = b \cdot a$$

Several properties

$$a + 1 = 1$$
$$a + 0 = a$$
$$a + a = a$$
$$a + \neg a = 1$$
$$a + b = b + a$$

$$a \cdot 1 = a$$
$$a \cdot 0 = 0$$
$$a \cdot a = a$$
$$a \cdot \neg a = 0$$
$$a \cdot b = b \cdot a$$

$$\neg\neg a = a$$
$$\neg(a + b) = \neg a \cdot \neg b$$
$$\neg(a \cdot b) = \neg a + \neg b$$

Distributive law

$$a \cdot (b + c) = a \cdot b + a \cdot c$$
$$a + b \cdot c = (a + b) \cdot (a + c)$$

Several properties

$$a + 1 = 1$$
$$a + 0 = a$$
$$a + a = a$$
$$a + \neg a = 1$$
$$a + b = b + a$$

$$a \cdot 1 = a$$
$$a \cdot 0 = 0$$
$$a \cdot a = a$$
$$a \cdot \neg a = 0$$
$$a \cdot b = b \cdot a$$

$$\neg\neg a = a$$
$$\neg(a + b) = \neg a \cdot \neg b$$
$$\neg(a \cdot b) = \neg a + \neg b$$
$$a + a \cdot b = a$$
$$a \cdot (a + b) = a$$

Distributive law

$$a \cdot (b + c) = a \cdot b + a \cdot c$$
$$a + b \cdot c = (a + b) \cdot (a + c)$$

Given a set $\mathbb{B} = \{0, 1\}$. The syntax of Boolean function is defined as follows

$$f :: x \mid \neg(f) \mid f_1 \wedge f_2 \mid f_1 \vee f_2$$

where x is a variable in $\mathbb{B}$.

Given a set $\mathbb{B} = \{0, 1\}$. The syntax of Boolean function is defined as follows

$$f :: x \mid \neg(f) \mid f_1 \wedge f_2 \mid f_1 \vee f_2$$

where x is a variable in $\mathbb{B}$.
Example:

- $f(x_1, x_2, x_3) = x_1x_2 + \neg x_1x_3 + x_2\neg x_3$
- $g(x_1, x_2, x_3) = x_1 + x_2 + x_2x_3$
- $h(x_1, x_2, x_3) = 1$

Given a Boolean function $f(x_1, x_2, \ldots, x_n)$.

Given a Boolean function $f(x_1, x_2, \ldots, x_n)$.

- A tuple $(i_1, i_2, \ldots, i_n) \in \mathbb{B}^n$ model for f if $f(i_1, i_2, \ldots, i_n) = 1$

Given a Boolean function $f(x_1, x_2, \ldots, x_n)$.

- A tuple $(i_1, i_2, \ldots, i_n) \in \mathbb{B}^n$ model for f if $f(i_1, i_2, \ldots, i_n) = 1$
- A Boolean function $f(x_1, x_2, \ldots, x_n)$ is called satisfiable if it has at least one model

Given a Boolean function $f(x_1, x_2, \ldots, x_n)$.

- A tuple $(i_1, i_2, \ldots, i_n) \in \mathbb{B}^n$ model for f if $f(i_1, i_2, \ldots, i_n) = 1$
- A Boolean function $f(x_1, x_2, \ldots, x_n)$ is called satisfiable if it has at least one model
- A Boolean function $f(x_1, x_2, \ldots, x_n)$ is called non-satisfiable if it has no model

Given a Boolean function $f(x_1, x_2, \ldots, x_n)$.

- A tuple $(i_1, i_2, \ldots, i_n) \in \mathbb{B}^n$ model for f if $f(i_1, i_2, \ldots, i_n) = 1$
- A Boolean function $f(x_1, x_2, \ldots, x_n)$ is called satisfiable if it has at least one model
- A Boolean function $f(x_1, x_2, \ldots, x_n)$ is called non-satisfiable if it has no model

The function $f(x_1, x_2, x_3) = (x_1x_2 + x_1\neg x_2)x_3$ is satisfiable with one model being $(1, 0, 1)$

Given a Boolean function $f(x_1, x_2, \ldots, x_n)$.

- A tuple $(i_1, i_2, \ldots, i_n) \in \mathbb{B}^n$ model for f if $f(i_1, i_2, \ldots, i_n) = 1$
- A Boolean function $f(x_1, x_2, \ldots, x_n)$ is called satisfiable if it has at least one model
- A Boolean function $f(x_1, x_2, \ldots, x_n)$ is called non-satisfiable if it has no model

The function $f(x_1, x_2, x_3) = (x_1x_2 + x_1\neg x_2)x_3$ is satisfiable with one model being $(1, 0, 1)$

The function $g(x_1, x_2) = (x_1x_2 + \neg x_1\neg x_2)(x_1\neg x_2 + \neg x_1x_2)$ is non-satisfiable

- Two Boolean functions f and g are called equisatisfiable if both are satisfiable or both are non-satisfiable.

- Two Boolean functions f and g are called equisatisfiable if both are satisfiable or both are non-satisfiable.

$$f = (x + y) \qquad g = (x \vee z) \wedge (y \vee \neg z)$$

- Two Boolean functions f and g are called equisatisfiable if both are satisfiable or both are non-satisfiable.

$$f = (x + y) \qquad g = (x \vee z) \wedge (y \vee \neg z)$$

Models for f: $(0, 1), (1, 0), (1, 1)$
Models for g: $(0, 1, 1), (1, 0, 0), (1, 1, 0), (1, 1, 1)$

- A Boolean function is called an atom if it involves only a single variable and no operators: a

- A Boolean function is called an atom if it involves only a single variable and no operators: a
- A Boolean function is called a literal if it is either an atom or its negation: $a, \neg a$

- A Boolean function is called an atom if it involves only a single variable and no operators: a
- A Boolean function is called a literal if it is either an atom or its negation: $a, \neg a$
- A Boolean function is called a term if it is a conjunction of several literals: $a, ab, ab\neg c$

- A Boolean function is called an atom if it involves only a single variable and no operators: a
- A Boolean function is called a literal if it is either an atom or its negation: $a, \neg a$
- A Boolean function is called a term if it is a conjunction of several literals: $a, ab, ab\neg c$
- A Boolean function is called a clause if it is a disjunction of several literals: $a + b, a + b + \neg c$

- A Boolean function is called an atom if it involves only a single variable and no operators: a
- A Boolean function is called a literal if it is either an atom or its negation: $a, \neg a$
- A Boolean function is called a term if it is a conjunction of several literals: $a, ab, ab\neg c$
- A Boolean function is called a clause if it is a disjunction of several literals: $a + b, a + b + \neg c$
- A Boolean function is in Conjunctive Normal Form (CNF) if it is a conjunction of several clauses: $a + b, (a + b)(c + \neg d)$

- A Boolean function is called an atom if it involves only a single variable and no operators: a
- A Boolean function is called a literal if it is either an atom or its negation: $a, \neg a$
- A Boolean function is called a term if it is a conjunction of several literals: $a, ab, ab\neg c$
- A Boolean function is called a clause if it is a disjunction of several literals: $a + b, a + b + \neg c$
- A Boolean function is in Conjunctive Normal Form (CNF) if it is a conjunction of several clauses: $a + b, (a + b)(c + \neg d)$
- A Boolean function is in Disjunctive Normal Form (DNF) if it is a disjunction of several terms: $ab, ab + c\neg d$

- A Boolean function is called an atom if it involves only a single variable and no operators: a
- A Boolean function is called a literal if it is either an atom or its negation: $a, \neg a$
- A Boolean function is called a term if it is a conjunction of several literals: $a, ab, ab\neg c$
- A Boolean function is called a clause if it is a disjunction of several literals: $a + b, a + b + \neg c$
- A Boolean function is in Conjunctive Normal Form (CNF) if it is a conjunction of several clauses: $a + b, (a + b)(c + \neg d)$
- A Boolean function is in Disjunctive Normal Form (DNF) if it is a disjunction of several terms: $ab, ab + c\neg d$
- A Boolean function in CNF form can be converted into DNF form; and vice versa.

$$
\begin{aligned}
(a+b)(a+c+d)(b+\neg d) &\equiv (a+ac+ad+ab+bc+bd)(b+\neg d) \\
&\equiv (a+bc+bd)(b+\neg d) \\
&\equiv ab+a\neg d+bc+bc\neg d+bd+0 \\
&\equiv ab+a\neg d+bc+bd
\end{aligned}
$$

$$
\begin{aligned}
(a+b)(a+c+d)(b+\neg d) &\equiv (a+ac+ad+ab+bc+bd)(b+\neg d) \\
&\equiv (a+bc+bd)(b+\neg d) \\
&\equiv ab+a\neg d+bc+bc\neg d+bd+0 \\
&\equiv ab+a\neg d+bc+bd
\end{aligned}
$$

$$
\begin{aligned}
a_1b_1+a_2b_2+\ldots+a_nb_n \equiv &(a_1+a_2+\ldots+a_n)(b_1+a_2+\ldots+a_n) \\
&(a_1+b_2+\ldots+a_n)\ldots \\
&\vdots \\
&(b_1+b_2+\ldots+a_n)(b_1+b_2+\ldots+b_n)
\end{aligned}
$$

$$
\begin{aligned}
(a+b)(a+c+d)(b+\neg d) &\equiv (a+ac+ad+ab+bc+bd)(b+\neg d) \\
&\equiv (a+bc+bd)(b+\neg d) \\
&\equiv ab+a\neg d+bc+bc\neg d+bd+0 \\
&\equiv ab+a\neg d+bc+bd
\end{aligned}
$$

$$
\begin{aligned}
a_1b_1+a_2b_2+\ldots+a_nb_n \equiv &(a_1+a_2+\ldots+a_n)(b_1+a_2+\ldots+a_n) \\
&(a_1+b_2+\ldots+a_n)\ldots \\
&\vdots \\
&(b_1+b_2+\ldots+a_n)(b_1+b_2+\ldots+b_n)
\end{aligned}
$$

In general, converting CNF to DNF is easier than converting DNF to CNF.

p	q	r	$(p \vee \neg q) \rightarrow r$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

p	q	r	$(p \vee \neg q) \rightarrow r$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$$(p \vee \neg q) \rightarrow r \equiv \neg p \neg q r + \neg p q \neg r + \neg p q r + p \neg q r + p q r$$

p	q	r	$(p \vee \neg q) \rightarrow r$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$$(p \vee \neg q) \rightarrow r \equiv \neg p \neg q r + \neg p q \neg r + \neg p q r + p \neg q r + p q r$$
$$\equiv (p + q + r)(\neg p + q + r)(\neg p + \neg q + r)$$

How to convert to CNF

How to convert to CNF

- Truth table (exponential)

How to convert to CNF

- Truth table (exponential)
- Simplification, distributive, DeMorgan (exponential)

How to convert to CNF

- Truth table (exponential)
- Simplification, distributive, DeMorgan (exponential)
- Tseitin encoding (linear)
  produces a Boolean function that is equisatisfiable

How to convert to CNF

- Truth table (exponential)
- Simplification, distributive, DeMorgan (exponential)
- Tseitin encoding (linear)
  produces a Boolean function that is equisatisfiable
  - Define new variables for each subformula

How to convert to CNF

- Truth table (exponential)
- Simplification, distributive, DeMorgan (exponential)
- Tseitin encoding (linear)
  produces a Boolean function that is equisatisfiable
  - Define new variables for each subformula
  - Define new clauses for each variable

How to convert to CNF

- Truth table (exponential)
- Simplification, distributive, DeMorgan (exponential)
- Tseitin encoding (linear)
  produces a Boolean function that is equisatisfiable
  - Define new variables for each subformula
  - Define new clauses for each variable
  - Apply the following rules

How to convert to CNF

- Truth table (exponential)
- Simplification, distributive, DeMorgan (exponential)
- Tseitin encoding (linear)
  produces a Boolean function that is equisatisfiable
  - Define new variables for each subformula
  - Define new clauses for each variable
  - Apply the following rules

$$p \leftrightarrow (a \vee b) \equiv (p \vee \neg a) \wedge (p \vee \neg b) \wedge (\neg p \vee a \vee b)$$
$$p \leftrightarrow (a \wedge b) \equiv (\neg p \vee a) \wedge (\neg p \vee b) \wedge (p \vee \neg a \vee \neg b)$$
$$p \leftrightarrow \neg a \equiv (p \vee a) \wedge (\neg p \vee \neg a)$$

$$f = ((p \vee q) \wedge r) \vee \neg p$$

$$f = ((p \vee q) \wedge r) \vee \neg p$$

$$x_1 := p \vee q$$

$$f = ((p \vee q) \wedge r) \vee \neg p$$

$$x_1 := p \vee q$$

$$x_2 := (p \vee q) \wedge r = x_1 \wedge r$$

$$f = ((p \vee q) \wedge r) \vee \neg p$$

$$x_1 := p \vee q$$

$$x_2 := (p \vee q) \wedge r = x_1 \wedge r$$

$$x_3 := \neg p$$

$$f = ((p \vee q) \wedge r) \vee \neg p$$

$$x_1 := p \vee q$$

$$x_2 := (p \vee q) \wedge r = x_1 \wedge r$$

$$x_3 := \neg p$$

$$x_4 := ((p \vee q) \wedge r) \vee \neg p = x_2 \vee x_3$$

$$f = ((p \vee q) \wedge r) \vee \neg p$$

$$x_1 := p \vee q$$
$$x_2 := (p \vee q) \wedge r = x_1 \wedge r$$
$$x_3 := \neg p$$
$$x_4 := ((p \vee q) \wedge r) \vee \neg p = x_2 \vee x_3$$

$$CNF(f) := x_4 \wedge (x_4 \leftrightarrow (x_2 \vee x_3)) \wedge (x_3 \leftrightarrow \neg p) \wedge (x_2 \leftrightarrow (x_1 \wedge r)) \wedge (x_1 \leftrightarrow (p \vee q))$$

$$f = ((p \vee q) \wedge r) \vee \neg p$$

$$x_1 := p \vee q$$
$$x_2 := (p \vee q) \wedge r = x_1 \wedge r$$
$$x_3 := \neg p$$
$$x_4 := ((p \vee q) \wedge r) \vee \neg p = x_2 \vee x_3$$

$$\begin{aligned} CNF(f) :=& x_4 \wedge (x_4 \leftrightarrow (x_2 \vee x_3)) \wedge (x_3 \leftrightarrow \neg p) \wedge \\ & (x_2 \leftrightarrow (x_1 \wedge r)) \wedge (x_1 \leftrightarrow (p \vee q)) \\ :=& x_4 \wedge (x_4 \vee \neg x_2) \wedge (x_4 \vee \neg x_3) \wedge (\neg x_4 \vee x_2 \vee x_3) \wedge \\ & (x_3 \vee p) \wedge (\neg x_3 \vee \neg p) \wedge \\ & (\neg x_2 \vee x_1) \wedge (\neg x_2 \vee r) \wedge (x_2 \vee \neg x_1 \vee \neg r) \wedge \\ & (x_1 \vee \neg p) \wedge (x_1 \vee \neg q) \wedge (\neg x_1 \vee p \vee q) \end{aligned}$$